



Requirements II — Interviewing, Observing and Prototyping

How to ask without inventing the answer, how to write down what you saw without deciding what it means, and how to provoke a reaction from someone who cannot describe what they want

Session 8 of 28 · Friday 2 October 2026 · 10:00 – 11:20 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

SESSION 8 OF 28

Requirements II — Interviewing, Observing and Prototyping

Friday, 2 October 2026 · 10:00 – 11:20 AM · Unit 2 · Requirements

LEARNING OBJECTIVES

- ✓ Plan a sprint's tasks so that they finish before the review and name what each one waits on.
- ✓ Build an interview from open, probing, confirming and referral questions — and repair a leading one.
- ✓ Write a three-column record that separates what you saw from what you think it means.
- ✓ Run a paper-prototype session that produces behaviour, not opinions.

FROM SESSION 7

Read, watch, ask, prototype — in that order, each instrument covering the blind spot of the one before. Today's work block decides the stack, then plans a paper prototype — BIA's registration form; TOURE's search-screen prototype was planned on Tuesday. The discussion afterwards is the craft those two sessions need.

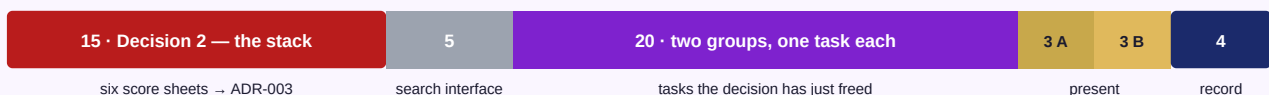
PART 1 · TEAM PROJECT WORK BLOCK

first 50 minutes of class

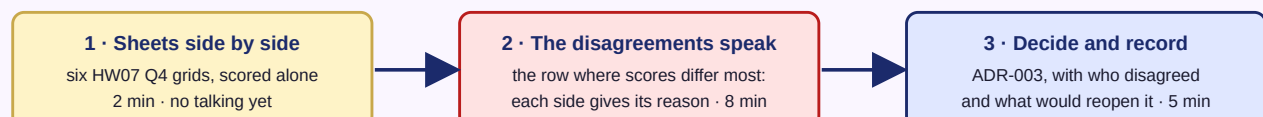
Decide the stack — then plan what it frees

On Tuesday the team toured the five technologies and three stacks, and decided nothing. Since then each student has scored the stacks **alone** (HW07 Q4). Today the six sheets become one decision, **ADR-003**. Sprint 1 starts on Monday, and four tasks cannot start without it. Scoring alone first matters: a team that talks first tends to converge on whoever spoke first.

THE FIFTY MINUTES



DECISION 2 · THE STACK — HOW THE FIFTEEN MINUTES RUN



The six questions each sheet scored, 1 to 3 per stack:

Six questions, each scored **1 to 3 for each stack**. “The six” means the six of you. Score a stack by its **weakest essential part**: a stack that is all Python except the one piece that saves data offline is not “all Python”.

Question	What it really asks — and why it matters here	3	2	1
1 Can every one of us review all of the code?	Rule 2 says nothing generated merges until a teammate who did not prompt it has read it line by line. If part of the code is in a language only one or two of you read (JavaScript, say), only they can review that part: every change waits for them, and when they are absent nobody can fix it. It is a question about the team, not the technology.	All of the team’s code is Python.	Python, plus a small, separate piece in something else — e.g. a form written in a spreadsheet.	An essential part — e.g. the tablet’s offline saving — is in a language most of you do not read.
2 Can the tablet do its whole job with no network, all day?	On outreach days the health worker takes the tablet to villages with no Wi-Fi and no mobile data , for eight hours. There it must register new patients, find existing ones (to avoid duplicates) and keep every record safe until it is back at the post to sync. A record lost in a village is a patient’s history lost.	Registers, searches and stores everything on the tablet itself; nothing is lost if the battery dies.	Registers offline, but searching existing patients is limited, <i>or</i> an ordinary action (clearing the browser) can lose unsent records.	A core function needs a connection — to the laptop or to the internet.
3 How much new tooling stands between us and the first saved record?	Sprint 1 ends on 16 October. Count what you must install and learn before one patient is saved on the tablet: a new build system, a new language, an account on an outside service. Each one is days of set-up nobody sees in the review.	Python and one or two libraries; first record within a few days.	One new toolchain (an Android build, or browser offline code); about a week.	Several new pieces, or one whose set-up nobody can predict.
4 Where does patient data live, and who else could see it?	These are health records (DR-02). Follow the arrows in the three diagrams: every place a record sits or passes through is a place it can leak or be lost. Data that never leaves the post’s own devices can be backed up by copying one file.	Only on devices at the post, in files the team controls and can copy.	At the post, but partly in browser storage the team cannot easily inspect or back up.	Passes through a server outside the post, run by someone else.
5 How well do frontier AI models know every part of this stack?	The AI will write most of the code, from your descriptions. It writes good code for tools with many years of public examples, and out-of-date code for tools whose interface changed recently — code that looks right and fails. Look at the “how well models know it” row of the capability table.	Every part is mainstream and stable (e.g. Flask, SQLite, FastAPI).	One part is popular but young or fast-changing — pin its version and check.	An essential part is niche; examples are scarce.
6 When it breaks in February and the team has gone, can the post recover?	After December nobody from the team is there. The clerk and the nurse have a one-page sheet. The fewer moving parts, accounts and passwords, the more likely a restart or a copied file fixes it.	One app and one database file: restart, or restore by copying a file.	Two parts that must both run (an app and a sync service; a browser and a server).	Depends on an outside service, account or password someone must keep alive.

The six are not equally important. Friday starts with one question: **for a rural health post, which rows matter most?**

The team decides; the instructor moderates. A tie is not settled by a vote: the team names the one question that matters most for a rural health post, and that row decides.

Every team decision is written down as a decision record — one page in [/arch/](#)

1. **Context** — what forced the decision now.
2. **Options** — at least two, each with what it costs and what it buys.
3. **Decision** — which one, in one sentence.
4. **Who decided, and who disagreed** — a team that records its disagreements can revisit them honestly.
5. **Revisit if...** — the event that would reopen it (for example, “the Android build fails the one-day trial”).

TWO GROUPS, ONE TASK EACH — TASKS THE DECISION HAS JUST FREED

Group A

AKAHI · TETE · BIA

- **S06 #1** registration paper prototype (BIA) — Wed 7 Oct
- Waits only on the field map (S05 #1), due today

Group B

TRAORE · TOURE · MANOUAN

- **S06 #4** ranking, `rank()` (TRAORE) — Fri 9 Oct
- Waited only on the stack — free from today

The owner writes; the other two challenge. During the first five minutes, TETE, TRAORE and TOURE agree the **search interface** at the board (decision D4), because three plans in two groups depend on it. The other owners write their plans after class, by **Monday 5 October**, using Tuesday's model plan (#3a), the S-16 example below and today's two plans as examples.

EVERY PLAN HAS SIX PARTS

1 The check

How we will know it is done: Given / When / Then, or what the stand-in clerk will see. It is the heart of what you give the AI.

2 Design choices

What you chose, what you rejected, one line why.

3 Steps

Numbered. Each a verb and an object, half a day or less.

4 Waits on / blocks

Including anything owned by the other group.

5 AI use

What you give the AI; which teammate reads the code before it merges.

6 Open question

For the Product Owner or the instructor — or “none”.

ONE MORE EXAMPLE OF A MODEL PLAN — A STORY NOBODY HOLDS (S-16, PRINT A PATIENT CARD)

Read on Tuesday; reread before planning today. S-16 is in the backlog but in no sprint yet, so nobody owns it. It is shown because it is the opposite of #3a in two ways: it is **code**, not a description; and its open question could **cancel or reshape the whole story**.

1 • The check	Given Awa KONE is registered as <code>BSM-L-26-00017</code> — village Azuretti, born 1994, mother Adjoua KONE — <i>when</i> the clerk opens her record on the desk laptop, network off, and presses “Print card”, <i>then</i> within 5 seconds a one-page PDF the size of the paper card opens, showing in the paper card’s order: post name, identifier in large type, family name, given names, sex, birth year, village, mother’s name — and nothing else. Also: pressing “Print card” again later prints the same identifier; and N’GUESSAN Adjoua Épiphanie prints with her apostrophe and accent correct. Why: “nothing else” is a privacy rule: the patient carries the card and others see it, so no diagnosis and no visit reasons. The accented name is the case that breaks PDF code most often. “The same identifier” guards against a reprint silently creating a new patient.
2 • Design choices	Chosen: make the PDF on the desk laptop with <code>fpdf2</code> — pure Python, one <code>pip install</code>, no internet, no system libraries — so it works with any of the three stacks, which all have Python on the laptop. Embed a Unicode TrueType font (DejaVu Sans), because <code>fpdf2</code>’s built-in fonts cannot print every character (the curly apostrophe ’, for one). Copy the layout of the paper card, so the clerk and patients recognise it. Rejected: reportlab (works, but far larger than a one-page card needs); an HTML page printed from the browser (the size and margins change with each browser and printer setting, so cards would come out different sizes); an online print or PDF service (needs internet, and sends patient data out of the post); a QR code on the card (nothing at the post can scan it — revisit if a scanner arrives). Why: each rejection names a concrete failure at <i>this</i> post: no internet, no scanner, a clerk who must recognise the card. Reasons tied to the setting are the ones that still hold in six months.
3 • Steps	0. Ask the open question (part 6) through the Product Owner — <i>before any other step</i>. 1. Photograph and measure the paper card; list its fields in order. 2. Write the check above into the story, with three synthetic patients: a short name, a long name, an accented name. 3. Give the AI the check, the Patient entry of the data dictionary, the field order and the constraints (<code>fpdf2</code>, the embedded font, returns the PDF as bytes, no network calls). It writes <code>render_card(patient)</code>. 4. Review: a teammate who did not prompt it reads it line by line (rule 2). 5. Add a “Print card” button to the patient screen that calls <code>render_card</code> and opens the PDF. 6. Check, network off: print the three cards, hold each next to the paper card, compare with the check, and record the result in the log. Why: step 0 costs ten minutes and can save every other step. Step 4 is separate from step 3 because the person who prompted the code is the worst person to review it.
4 • Waits on / blocks	Waits on: S05 #3a (the frozen identifier format, printed on the card); S-04 (a registered patient, and the patient screen the button goes on); ADR-003 (the screen framework for the button — though <code>render_card</code> itself does not wait on it). Blocks: nothing. Why: it waits on three things and blocks nothing, so it belongs <i>after</i> sprint 1. That nobody holds it is correct prioritisation, not neglect.
5 • AI use	The AI writes <code>render_card</code> from the check and the constraints in step 3 — nothing else is given to it. The reviewer checks four things: no network import (<code>requests</code>, <code>urllib</code>); the fields and their order match the check; no field beyond the card’s appears; the font is embedded. Why: giving the AI the check, rather than “make a patient card”, is rule 1 at work: the more precise the description, the less there is to fix. The reviewer’s four points come straight from the check and the design choices.

6 • Open question

- **For the Product Owner, to ask the nurse:** does the post have a working printer, paper and toner? *If no:* the story is rewritten, not built — “show the card on screen in large type, so the clerk copies it by hand onto a blank card” — a much smaller story.
- **For the instructor:** are patient cards issued by the health district? *If yes:* the post may not be allowed to print its own, and the story becomes “print the district card’s fields” or is dropped.

Why: the most valuable line in a plan is often the question that could cancel it — asked at step 0, before an hour of code.

THE PRESENTATIONS

Three minutes per group: **the check, the first step, and what it waits on.** Then the other group asks **one** question. The Scrum Master writes the question, the answer and any change to the plan in the log.

Output: ADR-003 in `arch/ADR-003-stack.md`, and two plans in `S08_progression.md`. The Scrum Master commits both **today**. TRAORE sets up the repository skeleton for the chosen stack (S05 #7) by Monday.

By Monday 5 October, evening: every other owner adds their six-part plan to the log.

Monday 5 October: sprint planning. The Product Owner confirms the sprint 1 forecast — S-01 to S-06, 29 points, a guess until the review (decision D5).

PART 2 Discussion — the last 30 minutes

Three ideas are discussed in class, about ten minutes each: sections marked **In class**. Everything marked **Read** is the reading assignment — read it before Tuesday.

1 • Three instruments, three ways to collect worthless data **In class**

Each instrument has one characteristic failure, and each failure produces data that *looks fine*. That is what makes them dangerous. Today is one remedy per failure.

Instrument	Its failure	What the spoiled data looks like	Remedy
Interview	The leading question	A confident “yes” — to your idea, not theirs	Question types, in order (§2)
Observation	The silent inference	A tidy conclusion with the evidence gone	The three-column record (§3)
Prototype	Asking for opinions	Comments on colour and layout	A task, then silence (§4)

2 · Interviewing without inventing the answer In class

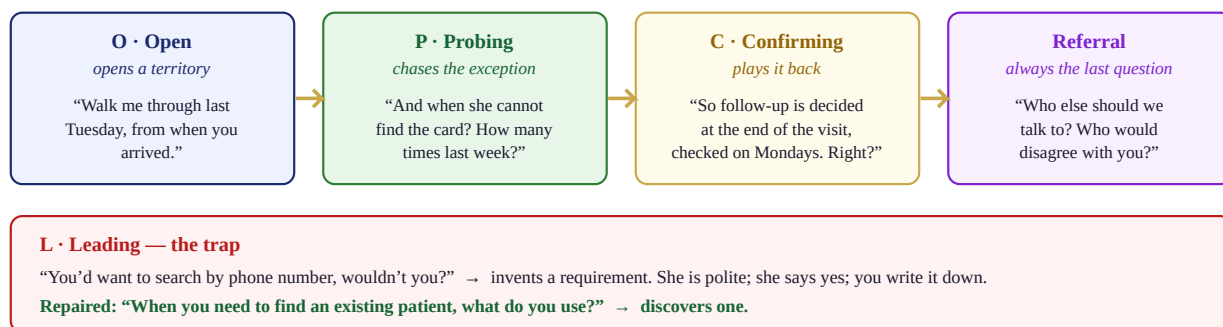


Figure 1. Four question types, roughly in this order. Corrections to your playback are worth more than agreements. The referral question is the cheapest in the course: it found the pharmacy assistant.

THE LEADING QUESTION IN ONE SENTENCE

A leading question is a hypothesis the interviewer hopes to confirm. The stakeholder is polite and wants you to succeed, so they agree — and nobody will ever know the finding was yours.

In class · repair it together

“You’d want the patient’s photo on the search results, wouldn’t you?” What does it invent? Rewrite it so that it discovers the same territory without supplying the answer.

APPLIED TO SANTE-BASSAM READ

A six-question protocol for the nurse-in-charge — typed, in order

#	Type	Question	Why it is there
1	O	Walk me through last Tuesday, from when you arrived until the first patient left.	Concrete and recent; opens the whole workflow
2	P	You said the clerk brings you the card. What happens when she cannot find it? How many times last week?	The exception behind S-05, with a frequency
3	O	Tell me about the notebook I saw beside the register.	The workaround — seen first, asked second
4	P	The last time the power went off during a consultation — what did you do with the record?	Data integrity in her words, with a real case
5	C	So: follow-up is decided at the end of the visit, written in the notebook, checked on Mondays. Right?	Playback; invites correction
6	R	Who else should we talk to? Who would disagree with you?	Referral

Leading · first draft

“Losing a record in a power cut would be unacceptable, right?”
Nobody says no — and it skips what she does about it today.

Repaired · Q4

“The last time the power went off — what did you do?” *“I write it on paper and copy it later. Twice I forgot.”* A requirement, its reason and a frequency.

3 • Separating what you saw from what you think it means In class

The commonest defect in a student record is the silent leap: you see the clerk check a number twice, you write “the system needs validation”, and what you actually saw is gone. The fix is mechanical — three columns, always.

Observation

What a camera would record. No *because*, no *needs*, no *struggles with*. Counts wherever you can get them.

Inference

What you think it means. Allowed to be wrong — that is what the third column is for.

How to test it

The question you will ask or the thing you will count. An untested inference is not evidence.

APPLIED TO SANTE-BASSAM

Four rows from one morning at the desk — reference record, rows 3, 4, 6 and 7 of seven

#	Observation	Inference	How to test it
3 ★	The nurse keeps a private notebook of patients to follow up.	The register does not support follow-up at all.	Ask what is in it and why (protocol Q3).
4 ★	Of 31 returning patients, 19 gave the child’s name and their own; the clerk found 17 by scanning for the mother; 2 were registered again.	The mother’s name is the real lookup key. Duplicates run at about 6% of returning visits.	Count duplicate-looking entries in 200 register rows.
6	Median time to register a new patient on paper: 75 s, over 9 registrations.	Anything slower will be abandoned.	Time the same task on the prototype, same clerk.
7	Power failed 11:20–11:50. Clerk continued on paper; two entries copied back later, one incompletely.	Partial writes, not total loss, are the realistic failure.	Inspect the register for incomplete rows.

In class • split it together

“The clerk struggles with the long form.” What would a camera have recorded? What is the inference? What would you count to test it — on BIA’s paper prototype next week?

LOOK FOR THE NOTEBOOK

Rows 3 and 4 are **workarounds**: somebody built a private system to survive a gap in the official one. A workaround is the strongest evidence you will find — the person has already paid for it with their own time. And row 4 carries a **number**: “19 of 31” is what moved S-05 up the backlog. An unquantified workaround is a story; a quantified one is a priority.

4 • Prototyping: provoking a reaction In class

Interviews and observation share one blind spot: neither shows how someone reacts to something that does not exist yet. For that, put an artifact in front of them.

DEFINITION

A **throwaway prototype** is an artifact built to be discarded, whose only purpose is to provoke a reaction that reveals a requirement. No code from it survives. Paper is usually right, because nobody mistakes paper for a finished product. **The rule:** give a task, then say nothing.

Find a patient

awa	go
----------------------------------	-----------------------------------

name	village	born	mother
Awa Kone	Modeste	2024	Mariam
Awa Kone	Vitré 2	2023	Fatou
Awa Diallo	Modeste	2019	Aminata
Awa Touré	Mondoukou	2022	Rokia
Awa Kouassi	Modeste	2021	Adjoua
... 14 results			

- 1 She typed **awa**, saw 14 results, and stopped.
First-name search alone is useless at this size.
- 2 She looked for a place to type the mother's name.
Mother must be a search field, not just a column.
- 3 Two identical names: "the one from Modeste", at once.
Village disambiguates; birth year does not.
- 4 She read three rows, then looked up at us.
Ranking matters more than completeness.

Nothing below the dashed line was ever read.

Figure 2. Prototype A, the S-05 search screen, drawn by hand in ten minutes, and four things one session showed. The task given was: "A woman has arrived with a two-year-old called Awa. No card. She is from Modeste. Find the child's record." Then silence.

In class · back to the work block

BIA (#1) and TOURE (#7) planned paper prototypes an hour ago. What is the exact task sentence each will give the clerk — and what must the rest of the team *not* say while she works?

Read Running the session, and what it bought

- 1 **Draw it in ten minutes, by hand.** No colours, no logo, no menu. If it took longer, it is too finished to criticise.
- 2 **Give a task, then be silent** — especially when she pauses.
- 3 **Record in three columns** as she works. One person operates the paper, one writes.
- 4 **Ask only afterwards**, and only about what you saw: "you started typing and stopped — what were you thinking?"

What one hour of paper bought.

S-05's acceptance criterion changed: village sorts first, not birth year. One hypothesis was refuted: nobody wants search by phone number, which two of you had assumed. And one new story appeared, unprompted: "*if she is not here, I write it on a paper and give it to the nurse*" — a referral path no interview would have produced.

Three ways a prototype session fails.

You talk — every sentence is a leading question. **It is too pretty** — nobody criticises what looks finished. **You ask for opinions** — "what do you think?" gets a view on layout; a task gets the workflow.

SESSION 8 TAKEAWAY

Each instrument spoils its data in one characteristic way. Interview in order — open, probe, confirm, refer — and never put the answer in the question. Record in three columns: what you saw, what you think it means, how you will find out. Prototype with a task and silence. And look for the notebook: a workaround with a number attached is a requirement and its priority.

5 · Homework · individual · due before Session 9 (Tuesday 6 October)

About thirty minutes, plus the reading. Short on purpose: sprint 1 starts Monday, and your project actions are the larger job.

TEMPLATE

Download **CSC4305_HW08_Template.md** from **CSC_4305_SWE-Assignments**, type between the `<!-- ANSWER -->` markers, **rename** it **CSC4305_HW08_FirstName_LastName.md** and **commit** it to `/homework/S08/` in **your own private repository**.

Markdown only. Drafting in Word is fine — convert at word2md.com and commit the `.md`.

Q1 Repair five leading questions. Say what each invents or assumes, then rewrite it to discover the same territory. (a) “You’d find it useful to have the report emailed automatically, wouldn’t you?” (b) “How much time does the duplicate problem waste?” (c) “Would you prefer a dropdown or a search box?” (d) “The current system is pretty frustrating, isn’t it?” (e) “Do you need offline access?”

Q2 Fix three broken records. Split each into observation, inference and test. (a) “The clerk is frustrated by the slow register.” (b) “Patients don’t bring their cards because they lose them.” (c) “The nurse needs a better way to track follow-ups.”

Q3 Design a prototype session for S-12, the nurse’s Monday follow-up list. Half a page: what you draw, the exact task sentence, what you watch for, and the one outcome that would convince you the story is wrong.

If no outcome could change your mind, you are not running an experiment.

Q4 Reading. The sections of this lecture marked Read (the six-question protocol, and running a prototype session), and Sommerville, *Software Engineering* 10th ed., §4.4–4.5. Optional: Beyer & Holtzblatt, *Contextual Design*, ch. 3.

Monday 5 October: sprint 1 begins with sprint planning. **From Tuesday:** every requirement must trace to a source — an observation row, an interview question, a regulation clause or a prototype session. A requirement without a source is a guess with an identifier.

Key terms

open question

probing question

confirming question

playback

referral question

leading question

observation vs inference

three-column record

workaround

throwaway prototype

contextual inquiry

Readings and references

- Sommerville, I. *Software Engineering*, 10th ed. — Chapter 4, §4.1–4.5.
- Beyer, H. & Holtzblatt, K. *Contextual Design*, 1998 — Chapter 3.

- Gottesdiener, E. *Requirements by Collaboration*, 2002.
- Snyder, C. *Paper Prototyping*, 2003 — running the session.